

SECTIONS

Executive Summary

Working Features

Bugs Found

Security Issues

Incomplete Features

UI Issues

Performance Issues

Critical Fixes

Cross-salon cancellations if a doctorId somehow leaks.

- **Fix:** Change to `appointmentModel.findOne({ _id: appointmentId, shopId: doctor.shopId })` where `doctor.shopId` is fetched from `req.body.docId`. The newer `stylistController.js` already does this correctly — use that pattern.

HIGH

BUG-06 · `authAdmin` middleware uses hardcoded env-var credential, not per-salon

`authAdmin.js` checks `token_decode.email` ~~≠~~ `process.env.ADMIN_EMAIL`. This is a global single-admin credential, not tied to any salon. Any request with a token for

this email bypasses all per-salon scoping.

- **Fix:** Deprecate the `/api/admin` route entirely. All salon admin operations should go through `/api/salon-admin` with the proper `authSalonAdmin` middleware.

BUG-07 · `getAvailableDates` /

MEDIUM `getAvailableSlots` fall back to any `SlotSettings`

`userController.js` line 181: `SlotSettings.findOne({shopId }) || await SlotSettings.findOne()`. If no slot settings exist for a new salon, it silently falls back to another salon's slot settings. A customer booking at a new salon sees the wrong opening hours.

- **Fix:** Remove the `|| await SlotSettings.findOne()` fallback. Return a sensible default object (9am–6pm Mon–Sat) inline if no settings exist, matching the pattern used in `salonAdminController.js getSalonSlotSettings()`.

BUG-08 · `shopInfo` feature flags read from `localStorage`, can be stale

MEDIUM

`SalonAdminContext.jsx` line 12–13: `useState(JSON.parse(localStorage.getItem('shopInfo') || 'null'))`. If Super Admin turns off a feature flag after the salon admin has already logged in, the sidebar still shows the feature link and the page guard (when added) reads the old cached value. The admin must log out and back in for the change to take effect.

- **Fix:** Call `fetchShopInfo()` on every login and on a short interval (or on page focus), and ensure the sidebar reacts to the live `shopInfo` state rather than the stale `localStorage` snapshot.

BUG-09 · `adminController.js`

MEDIUM `bookingController.js` still hardcode `shopId: 'SHOP001'`

`adminController.js` line 623: `shopId: 'SHOP001'` hardcoded in a notification. The legacy booking controller (the *Before slot booked* file) has similar issues. If these code paths are ever reached, they silently emit data to the wrong salon.

- **Fix:** Audit and remove all hardcoded `'SHOP001'` strings; replace with the dynamic `shopId` from the relevant appointment or doctor document.

3 · Security / Data Isolation Issues

CRITICAL

SEC-01 · `/api/admin/*` routes bypass per-salon isolation

Described in BUG-01. The global admin credential grants unscoped read/write access across all salons. Cross-salon data leakage is possible through `allDoctors`, `adminDashboard`, `getSlotSettings`, and others.

HIGH

SEC-02 · Customer phone number is globally unique — PII leakage via signup error

If a customer has registered at Salon A and attempts to register at Salon B with the same phone, they receive "Phone number already registered." This leaks the fact that the number is in the system across salons. Described in BUG-02.

MEDIUM

SEC-03 · Public `/api/shop/:slug` returns full shop document

`shopRoute.js` line 57: `res.json({ success: true, shop })` returns the entire Mongoose document including `upiId`, `bankName`, `gstNumber`, `adminId`, and all feature flags.

These fields are not intended for public customer-facing views.

- **Fix:** Use `.select()` to return only the fields needed by the customer-facing UI: `shopName, slug, logo, address, city, state, phone, status, couponEnabled, packageEnabled, paymentIntegrationEnabled`. Sensitive financial and admin fields should only be returned to authenticated admin routes.

4 · Incomplete / Missing Features

INCOMPLETE Payment Integration (Razorpay / Stripe)

Routes `POST /api/user/payment-razorpay` and `POST /api/user/payment-stripe` exist and are called from `Appointment.jsx`. However, UPI screenshot-based payment is the only fully tested flow. Razorpay/Stripe require live API keys (`RAZORPAY_KEY_ID`, `STRIPE_SECRET_KEY`) in `.env` that may not be set in production. The payment verification callbacks have no webhook signature validation.

INCOMPLETE WhatsApp Cloud API (Broadcast — API mode)

The "API send" mode in `broadcastController.js` requires `WHATSAPP_API_TOKEN` and `WHATSAPP_PHONE_ID` env vars. Without them the feature gracefully rejects. The manual wa.me mode is fully functional. No message template pre-registration is handled (required for WhatsApp Business API for non-session messages).

INCOMPLETE Forgot Password (customer-facing)

`authRoutes.js` has OTP-based password reset routes. The frontend has a `ForgotPassword.jsx` page and `Verify.jsx`, but these are not linked from the ShopLogin page. Customers have no visible path to reset a forgotten password.

INCOMPLETE UPI QR Code Generation (Python dependency)

`shopRoute.js` `/upi-qr` spawns a Python process (`generate_qr.py`). This requires Python to be installed on the server. No fallback exists if Python is missing. If the salon's UPI QR is uploaded directly (via Cloudinary), this route is redundant, but the code will error loudly if called.

5 • UI Issues

AREA	ISSUE	SEVERITY
Create Salon form — Reset	The Cancel/Reset handler (line 148 of <code>CreateSalon.jsx</code>) does not reset <code>couponEnabled</code> ,	MEDIUM

button	<code>packageEnabled</code> , <code>stylistPanelEnabled</code> , or <code>broadcastEnabled</code> — only the non-toggle fields. After toggling flags and clicking Cancel, flags remain on.	
Broadcast page — Manual send opens many tabs	Sending to 50+ contacts opens 50+ browser tabs simultaneously. Most browsers will block this as a popup. The user gets no feedback on which tabs were blocked.	MEDIUM
Pending Salon approval — encoding artifacts	The middle-dot separator <code>.</code> and ellipsis <code>...</code> were corrupted to <code>Â·</code> and <code>â€¦</code> in a prior edit. These have been fixed in this session but indicate the file was saved as UTF-8 with BOM, which can reoccur.	LOW
Admin Sidebar — mobile bottom nav missing feature-gated items	The bottom navigation bar on mobile (<code>AdminSidebar.jsx</code> lines 191–213) hardcodes 5 links and does not show Billing, Coupons, Packages, or Broadcast even when those features are enabled. Desktop sidebar correctly hides/shows conditionally.	LOW
Dead pages in frontend	<code>About.jsx</code> , <code>Contact.jsx</code> , <code>Doctors.jsx</code> , <code>Home.jsx</code> , <code>Services.jsx</code> , <code>Login.jsx</code> , <code>ShopAdminPage.jsx</code> are present in <code>frontend/src/pages/</code> but not in any route in <code>App.jsx</code> . These are unreachable orphan files that inflate the bundle.	LOW

6 • Performance Issues

PERF

No MongoDB indexes on high-frequency query fields

`appointmentModel` is queried by `shopId`, `slotDate`,